

Audience: LEWG, SG14, WG21

Date: 2021-01-13

Project: Request for re-inclusion of std::hive proposal in C++23

Reply-to: Matthew Bentley <mattreecebentley@gmail.com>

REQUEST FOR RE-INCLUSION OF STD::HIVE PROPOSAL IN C++23

This is a brief rebuttal to the removal of P0447 from the C++23 agenda and a request for it's reinstatement after discussion within SG14. In the view of the SG14 committee, this paper has had significant investigation, insight and input over the years from both SG14 and LEWG members and can be considered 'mature' in the sense that the broad strokes will not change.

Secondly the core paper is not large and would not require significant amounts of LEWG time to discuss. The main aspect of the P0447 document which is large is the *appendices*, which take up over 1/3rd of the paper's length due to the many questions raised by both SG14 and LEWG members over the past 6 years which have required answers. Given that this type of container is a new concept to many committee members, this is not so unusual.

The implementation is also mature (6 years) and has had no issues reported for a very long time (https://github.com/mattreecebentley/plf_colony, forked to https://github.com/mattreecebentley/plf_hive for the exact-standards-matching version) despite being well-used. This is a good indication of stability.

The paper has only been discussed in a tentative way on the LEWG mailing list, since most of it's review period was taken up by discussion of the name (colony->hive) which is now considered resolved. What discussion was had raised several smaller issues, all of which have subsequently been addressed. I will now list these issues and their updated resolutions (summarised by Inbal Levi on the paper's github page [here](#)):

1. Recommendation to be careful not to over-specify the class, and leave room for future changes in implementation.
Author: This was considered, see also: "Is there active research in this problem space? Is it likely to change in future?" in paper's appendices.

Resolution: This section of the appendix has become increasingly detailed in subsequent versions of the paper. It is very difficult, if not impossible, to do this type of container while meeting the C++ container and iterator requirements, in a way other than has been specified in this paper. However points which have been identified as being over-specified,

such as the "skipfield type" parameter, have been altered or removed to reduce/eliminate specificity.

2. There was a discussion regarding having colony specified at `plf::` instead of `std::` to allow performance improvements which require ABI-breakage, the other side is that colony is usable by a large audience.

Resolution: As noted above, it is difficult/impossible to implement this type of container in a way that meets current C++ container/iterator requirements in a way other than what is outlined in the paper. See "Is there active research in this problem space? Is it likely to change in future?" in appendix D of the paper for more info about potential for ABI breakage, also "If this is for games, won't game devs just write their own versions for specific types in order to get a 1% speed increase anyway?". While being able to break ABI is always a good thing as far as potential for future improvement is concerned, the benefits of standardising are that better communication between fields and developers is possible and re-inventing of the wheel is negated. Multiple fields have their own version of this type of container, all with different names for it, as is discussed in the introduction of the paper.

3. Proposal suggests to reserve `std::limits`, consider reserving a smaller scope (possibly as a member in colony) or using `colony_limits`. Author: This was discussed broadly on SG14, making it a member creates ugly syntax, `colony_limits` is my preferred option.

Resolution: After mailing list discussion and the container's name change this was renamed to `std::hive_limits`.

4. Consider adding `constexpr` implementation.

Resolution: `constexpr` implementation would be trivial to implement in a future update to the standard, however I consider the field of `constexpr` containers somewhat emergent, with many unknowns and some issues in terms of non-`constexpr` performance, so am reluctant to make this container `constexpr` straight away. This is discussed in an appendix.

5. Elaborate on the purpose of parameter "Skipfield". Author: Meant to allow changes for this template instance which prioritize lowered memory usage over performance. Use of this type by an implementation is not guaranteed, may change this to `prioritize_performance` / `prioritize_memory_use` enum.

Resolution: this was changed to a scoped enum in subsequent versions of the paper and implementation, specifically `"hive_priority::performance"/"hive_priority::memory"`.

6. Consider making (`==`, `!=`, `<`, `>`) hidden friends (need to be in the class synopsis) / member functions (considering C++20 comparison rewrite rules) . (reference: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1601r0.pdf>)
Note by Jonathan Wakely: We have a proposal to support comparisons between containers with different types. I will update it for `<=>` comparisons. For now assume you don't need to overload for different priorities.

Resolution: This has been done, and the `<=>` operator for the container has also been created. The technical specification would benefit from subsequent inspection by LEWG.

7. Consider specifying `op==` in terms of the `std::equal`
Author: None of the other containers appear to do this, but I'm happy to do so if there's a

consensus.

Resolution: There didn't appear to be any additional support for this, so it has remained in keeping with other container's technical specifications.

8. A broad discussion regarding: advance/next/prev, result is they are specified as hidden friends of the iterator classes.

Resolution: This was implemented.

9. Consider removing member sort()

Author: Explanation under Design Decisions->Additional notes on specific functions->void sort(). To date there's been no interest in standardising the sort algorithm which colony reference implementation uses as a non-member.

Resolution: While the algorithm in question allows any non-random-access container to use sort techniques requiring random access, there was no interest in standardising this. In this context it is better to retain the sort function but leave the specific sort technique unspecified, save from mentioning that the sort technique is allowed to allocate. This is consistent with the approach taken by list<>.

10. Consider allowing different iter types in range insert and to constrain those types with an appropriate requires clause (to allow sentinel) Author: I think it's probably reasonable to expect that a new container should match functionality with the old ones in order to be a drop-in replacement in older code.

Resolution: I didn't really understand what the questioner was talking about at the time. At any rate, later versions have resolved this by supporting both mixed iterator types and sentinels for range insert/assign/constructors.

11. Remove duplication of paragraph which is included in memory()

Resolution: This was fixed.

12. Bike-shedding is welcome for reshape()

Resolution: No alternative names were suggested, so this has stayed as reshape() for the moment. Some people have subsequently suggested that something like change_block_capacity_limits would be more clear.

In addition, the container is of considerable benefit to multiple fields, as is outlined in the paper itself, and its inclusion in the standard sooner rather than later benefits the C++ worldwide community.

In summary, we request that hive be put back on the agenda for C++23. There are other papers which are much longer but have had considerably less oversight, which are now in progress for C++23, and we consider this egregious.